

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
дисциплины «Искусственный интеллект и машинное обучение»

Выполнил:
Хубиев Роберт Эльбрусович
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Руководитель практики:
Воронкин Роман Александрович,
доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Основы работы с библиотекой matplotlib

Цель: исследовать базовые возможности библиотеки matplotlib языка программирования Python

Ссылка на репозиторий: <https://github.com/truebobsuncle/MOII3>

Ход работы

Создали общедоступный репозиторий:

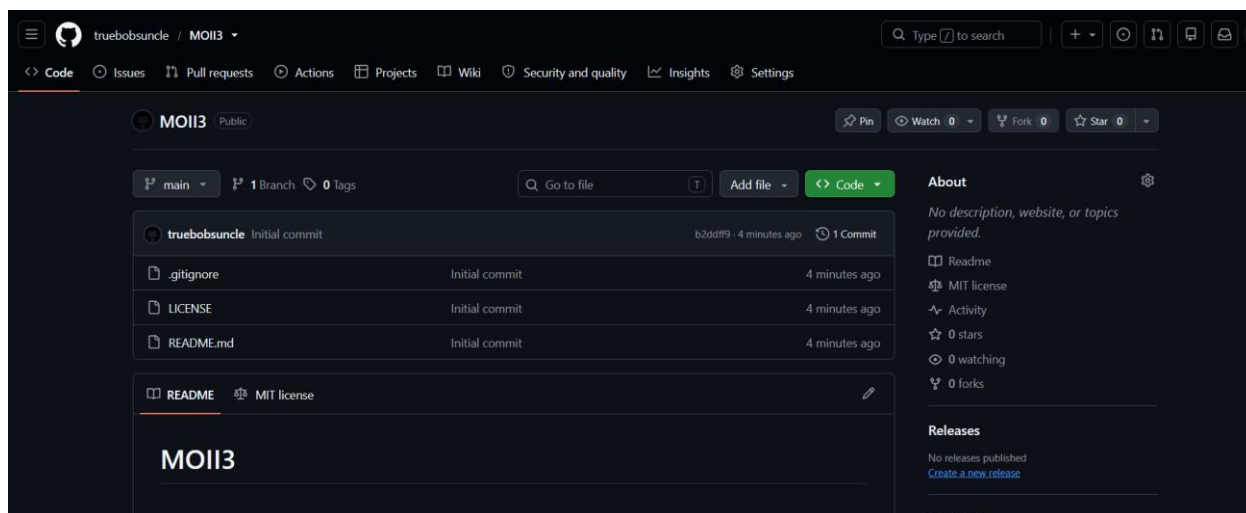


Рисунок 1. Репозиторий

Проработали примеры:

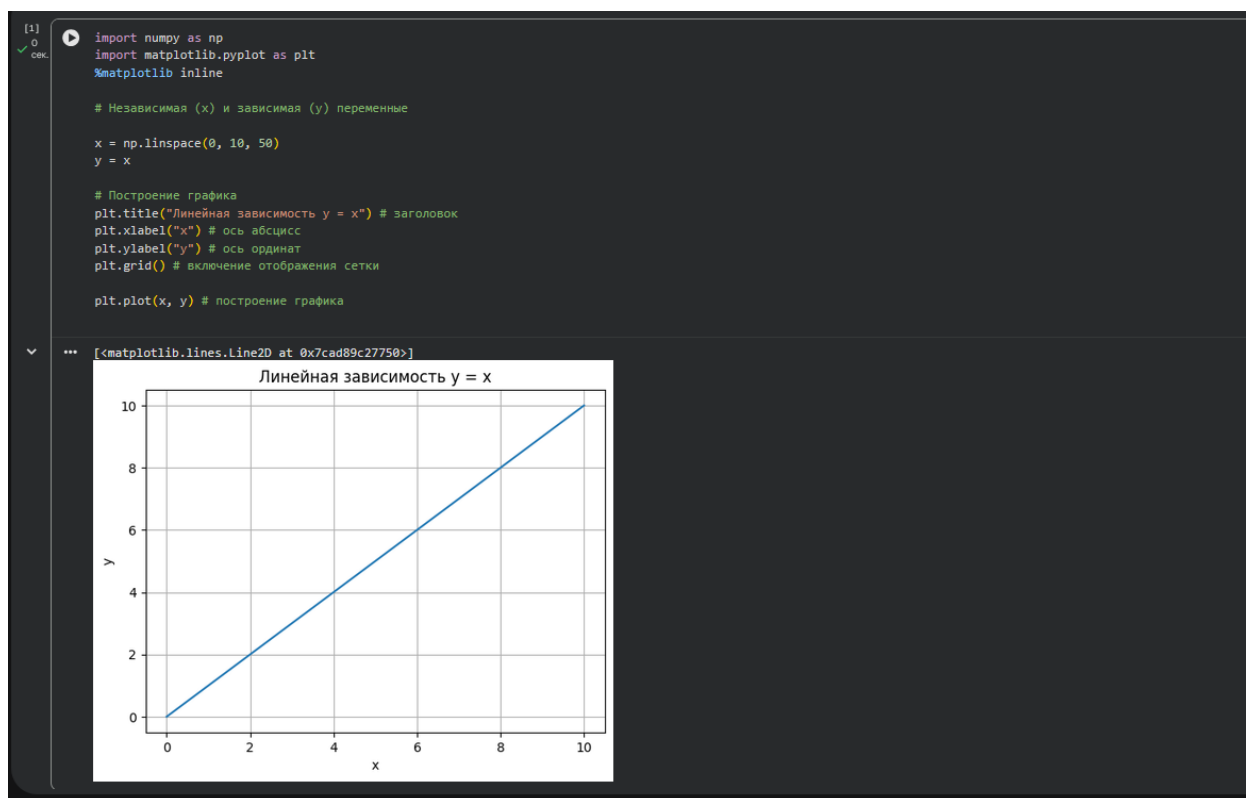


Рисунок 2. Пример 1

```

# Линейная зависимость
x = np.linspace(0, 10, 50)
y1 = x

# Квадратичная зависимость
y2 = [i**2 for i in x]

# Построение графика
plt.title("Зависимости: y1 = x, y2 = x^2") # заголовок
plt.xlabel("x") # ось абсцисс
plt.ylabel("y1, y2") # ось ординат
plt.grid() # включение отображения сетки

plt.plot(x, y1, x, y2) # построение графика

```

```

... [<matplotlib.lines.Line2D at 0x7cad89a13750>,
     <matplotlib.lines.Line2D at 0x7cad89a13890>]

```

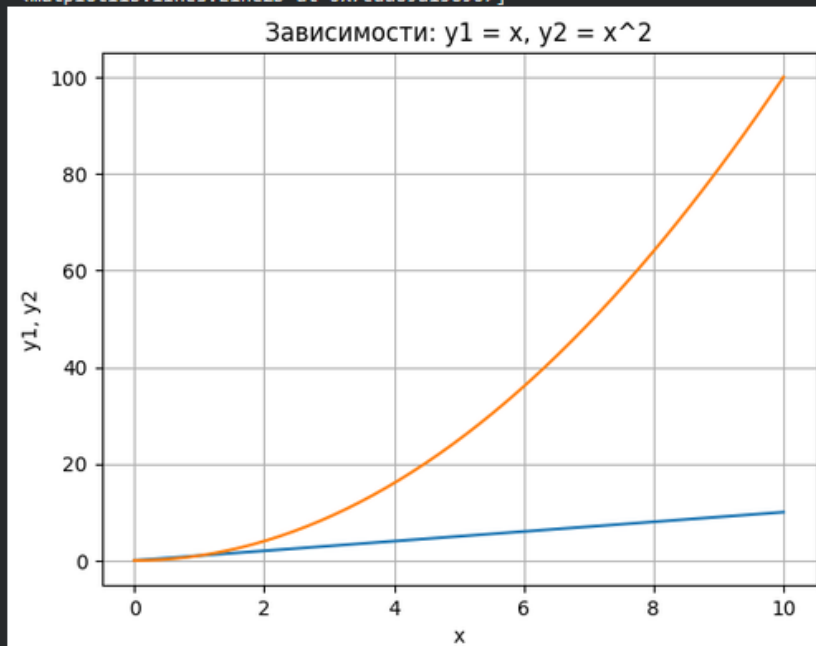


Рисунок 3. Пример 2

```

# Линейная зависимость
x = np.linspace(0, 10, 50)
y1 = x

# Квадратичная зависимость
y2 = [i**2 for i in x]

plt.figure(figsize=(9, 9))

plt.subplot(2, 1, 1)
plt.plot(x, y1)

plt.title("Зависимости: y1 = x, y2 = x^2")
plt.ylabel("y1", fontsize=14)

plt.grid(True)

plt.subplot(2, 1, 2)
plt.plot(x, y2)

plt.xlabel("x", fontsize=14)
plt.ylabel("y2", fontsize=14)
plt.grid(True)

```

Рисунок 4. Пример 3

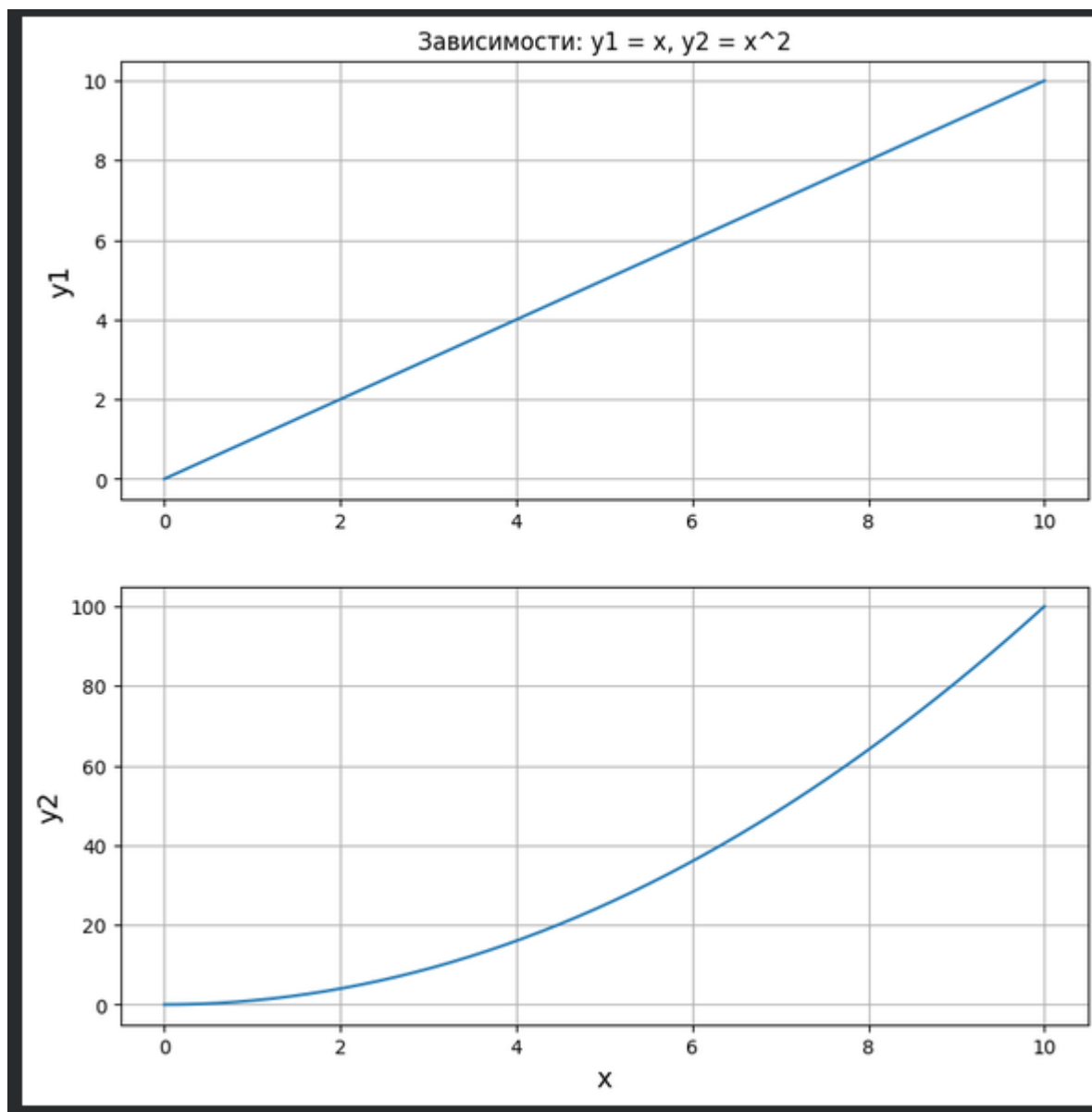


Рисунок 5. Результат выполн кода примера 3

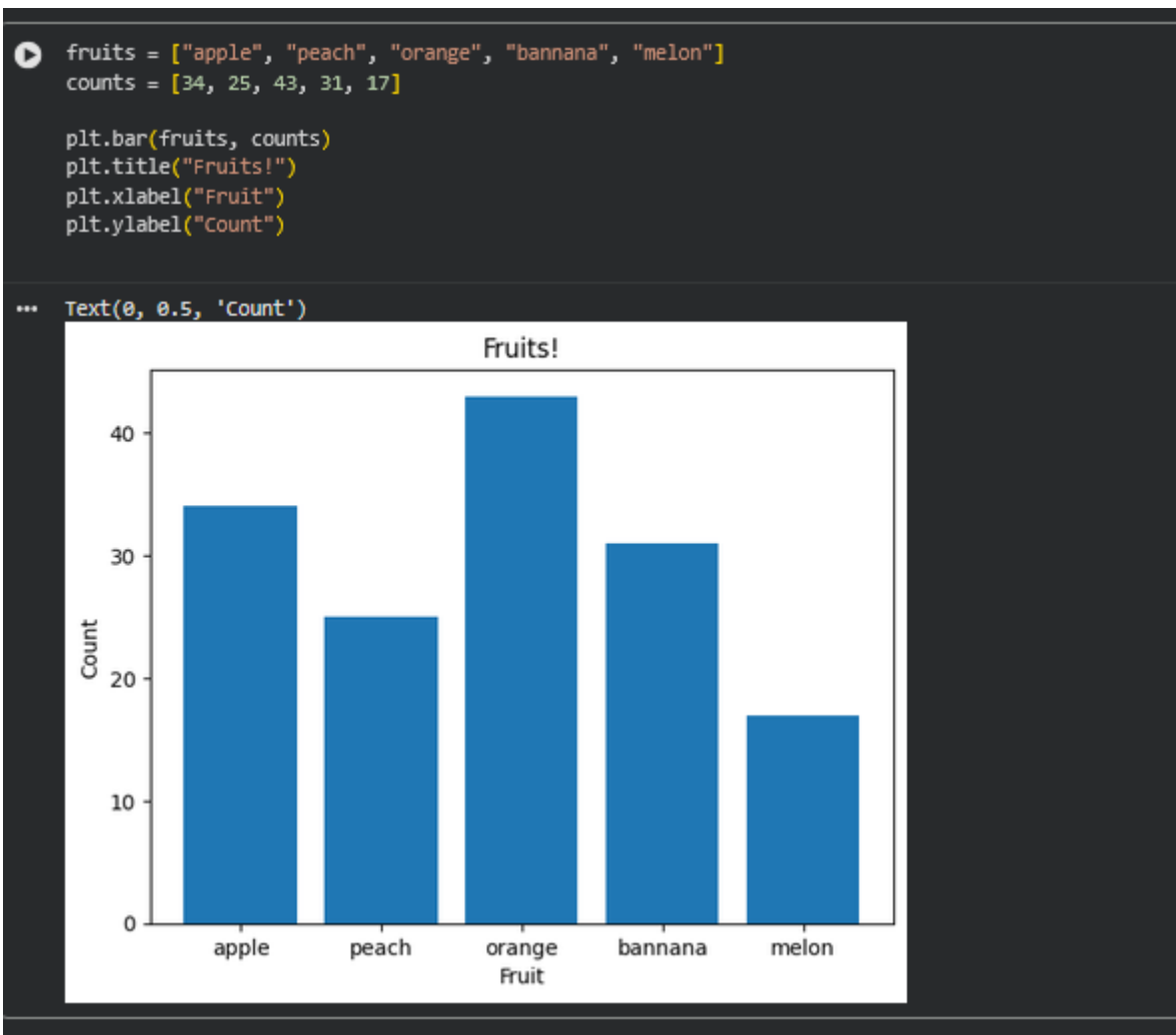


Рисунок 6. Пример 4

```
x = [1, 5, 10, 15, 20]
y1 = [1, 7, 3, 5, 11]
y2 = [4, 3, 1, 8, 12]

plt.figure(figsize=(12, 7))
plt.plot(x, y1, 'o-r', alpha=0.7, label="first", lw=5, mec='b', mew=2, ms=10)
plt.plot(x, y2, 'v-.g', label="second", mec='r', lw=2, mew=2, ms=12)

plt.legend()
plt.grid(True)
```

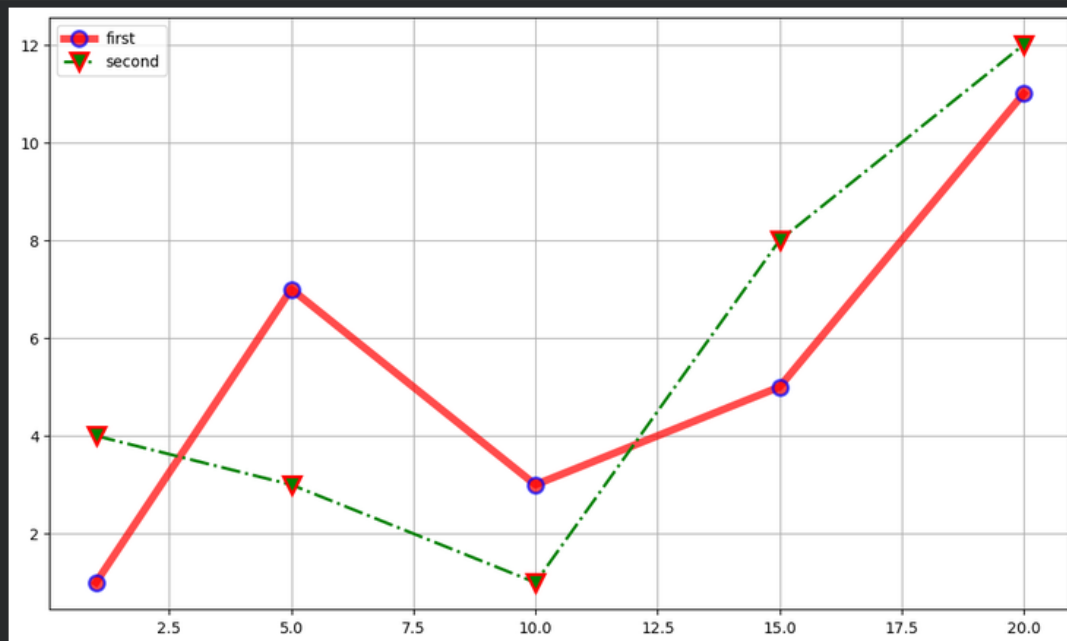


Рисунок 7. Пример 5

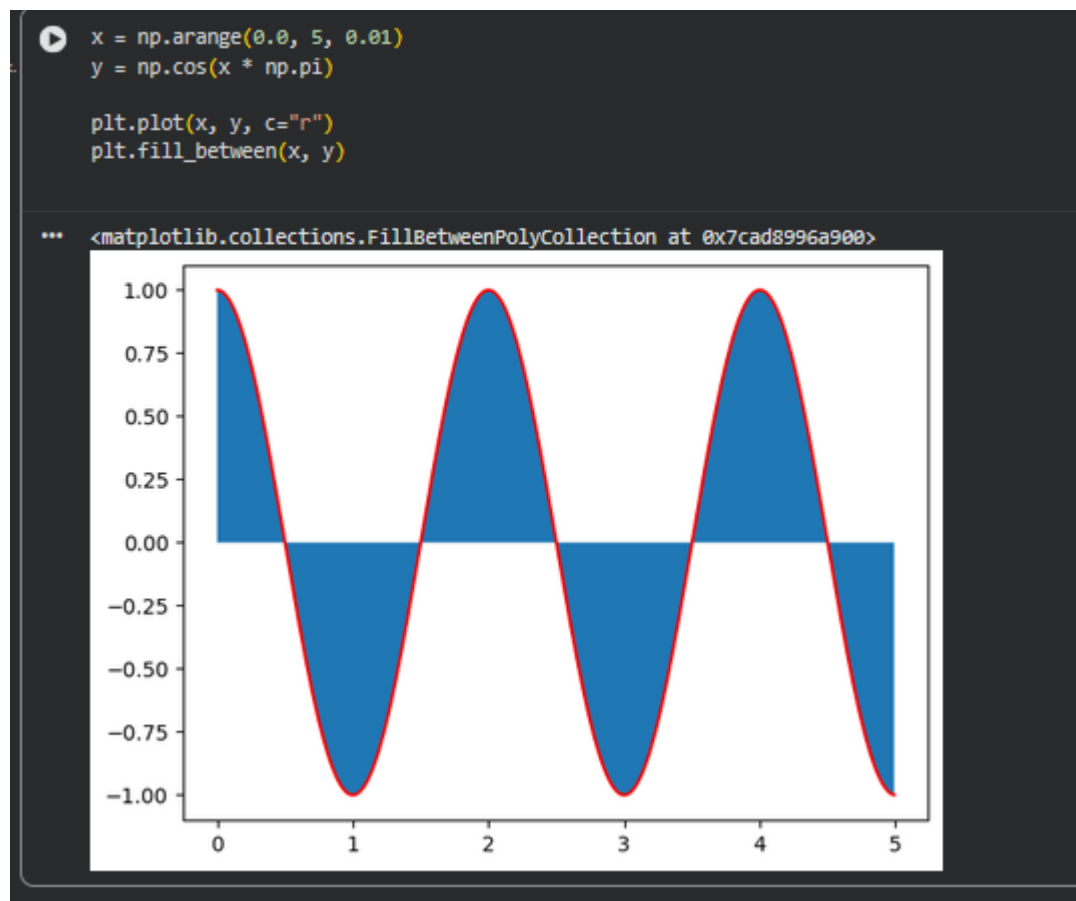


Рисунок 8. Пример 6

Практическая Работа:

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

x = np.linspace(-10, 10, 1000)
y = x ** 2

plt.figure(figsize=(8, 6))
plt.plot(x, y)

plt.title("График функции  $y = x^2$ ")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)

plt.show()
```

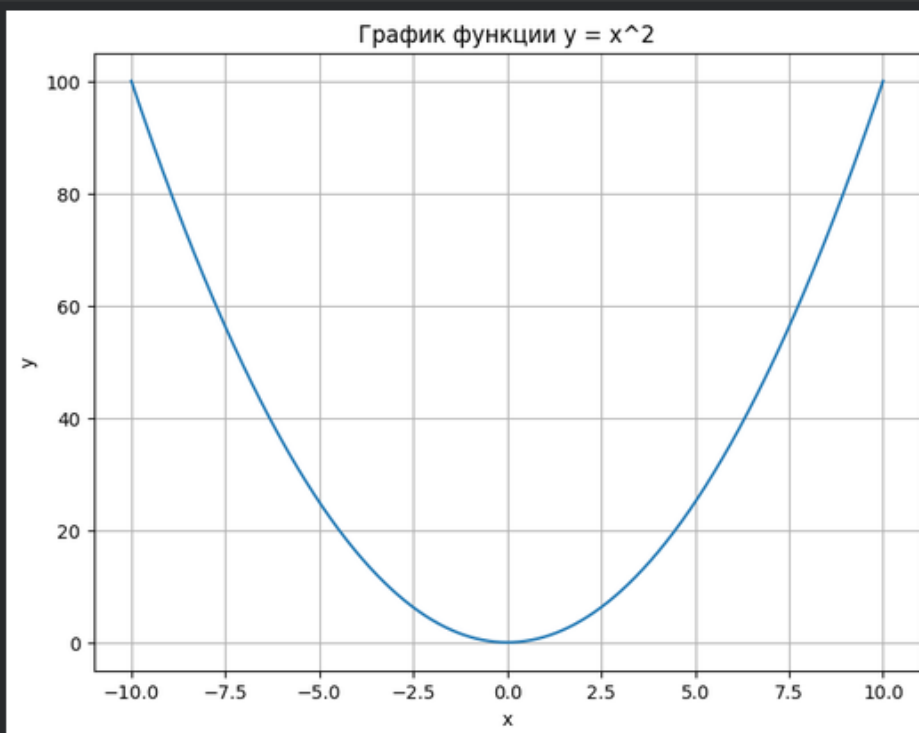


Рисунок 9. Код и результат выполнения 1 задания


```

x = np.linspace(-10, 10, 1000)

y1 = x
y2 = x ** 2
y3 = x ** 3

plt.figure(figsize=(8, 8))

plt.plot(x, y1, linestyle="--", color="blue", label="y = x")
plt.plot(x, y2, linestyle="--", color="green", label="y = x^2")
plt.plot(x, y3, linestyle="--", color="red", label="y = x^3")

plt.title("Графики функций y = x, y = x^2, y = x^3")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)

limit = 10
plt.xlim(-limit, limit)

```

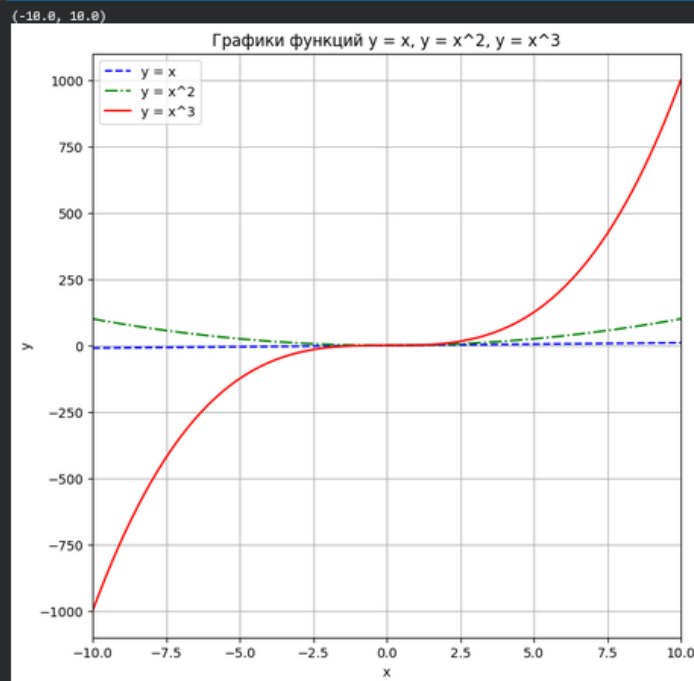


Рисунок 10. Код и результат выполнения 2 задания

```

np.random.seed(42)
x = np.random.rand(50)
y = np.random.rand(50)

sizes = 300 * y + 20

plt.figure(figsize=(8, 6))

scatter = plt.scatter(
    x,
    y,
    c=x,
    s=sizes,
    cmap="viridis",
    alpha=0.7
)

plt.colorbar(scatter, label="Значение x")
plt.title("Диаграмма рассеяния")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)

plt.show()

```

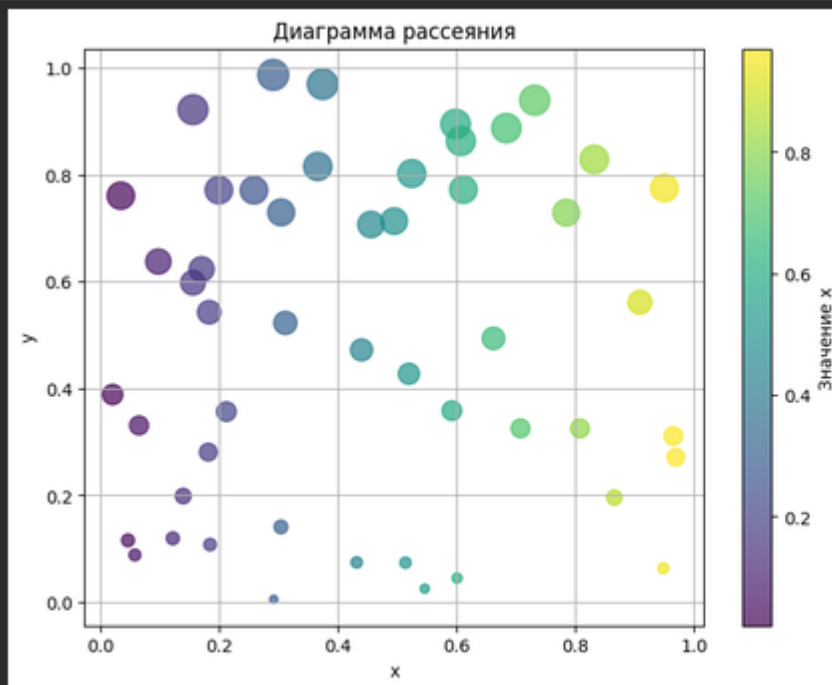


Рисунок 11. Код и результат выполнения 3 задания

```

np.random.seed(42)
data = np.random.normal(loc=0, scale=1, size=1000)

mean_value = np.mean(data)

plt.figure(figsize=(8, 6))

plt.hist(data, bins=30, color="skyblue", edgecolor="black")
plt.axvline(mean_value, color="red", linestyle="--", linewidth=2,
            label=f"Среднее = {mean_value:.2f}")

plt.title("Гистограмма нормального распределения")
plt.xlabel("Значение")
plt.ylabel("Частота")
plt.legend()
plt.grid(True)

plt.show()

```

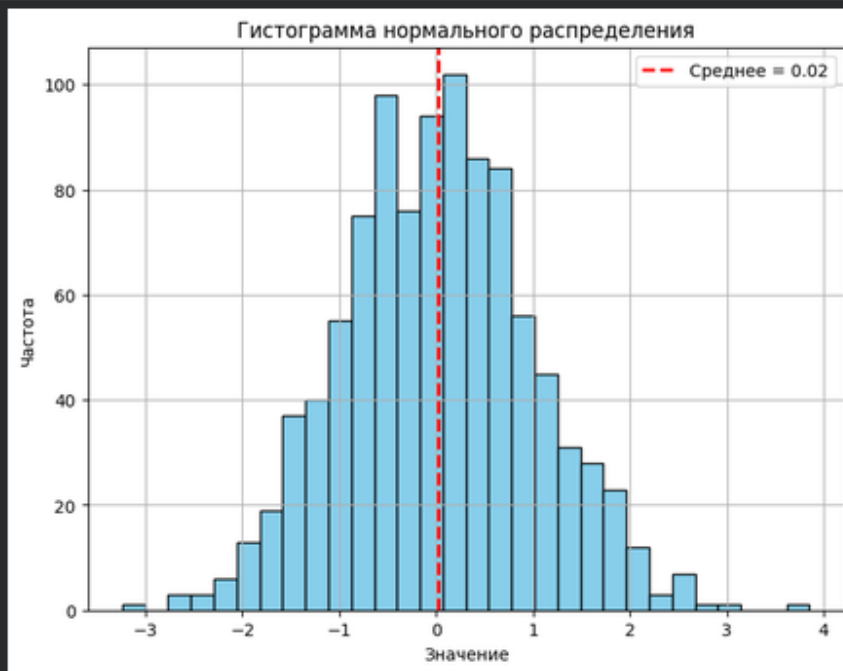


Рисунок 12. Код и результат выполнения 4 задания

```

grades = ["Отлично", "Хорошо", "Удовлетворительно", "Неудовлетворительно"]
counts = [20, 35, 30, 15]

plt.figure(figsize=(8, 6))

plt.bar(grades, counts, color=["green", "blue", "orange", "red"])

plt.title("Распределение оценок студентов")
plt.xlabel("Оценка")
plt.ylabel("Количество студентов")
plt.grid(axis="y")

plt.show()

```

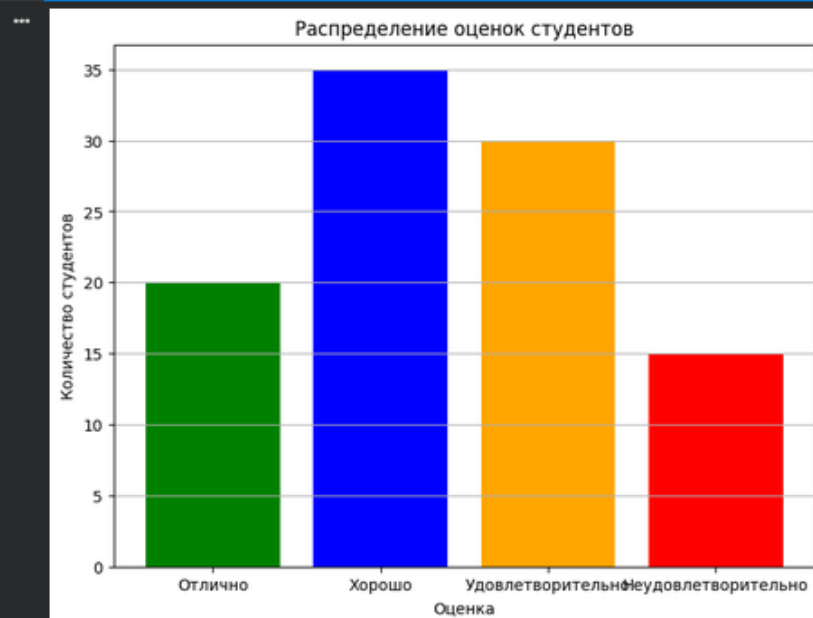


Рисунок 13. Код и результат выполнения 5 задания

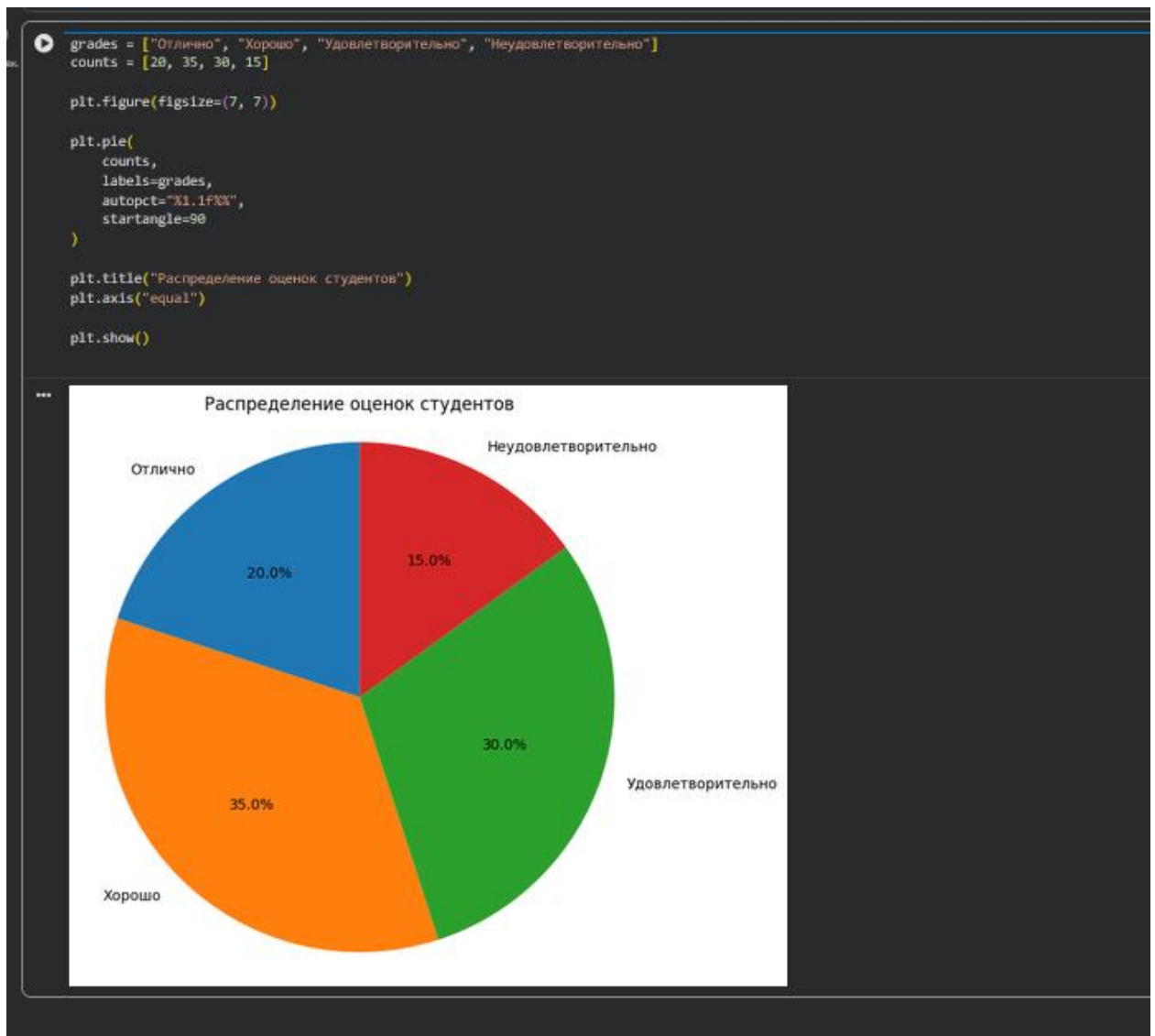


Рисунок 14. Код и результат выполнения 6 задания

```

x = np.linspace(-5, 5, 200)
y = np.linspace(-5, 5, 200)
x_grid, y_grid = np.meshgrid(x, y)

z = np.sin(np.sqrt(x_grid ** 2 + y_grid ** 2))

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection="3d")

surface = ax.plot_surface(
    x_grid,
    y_grid,
    z,
    cmap="viridis"
)

ax.set_title("3D-график  $z = \sin(\sqrt{x^2 + y^2})$ ")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")

fig.colorbar(surface, shrink=0.5, aspect=10)

plt.show()

```

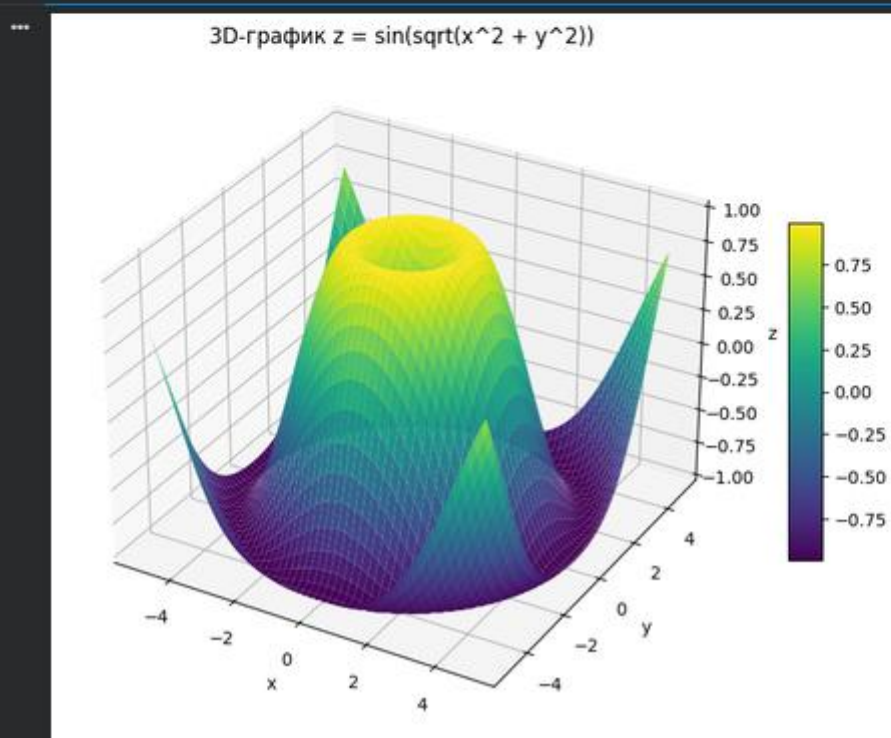


Рисунок 15. Код и результат выполнения 7 задания

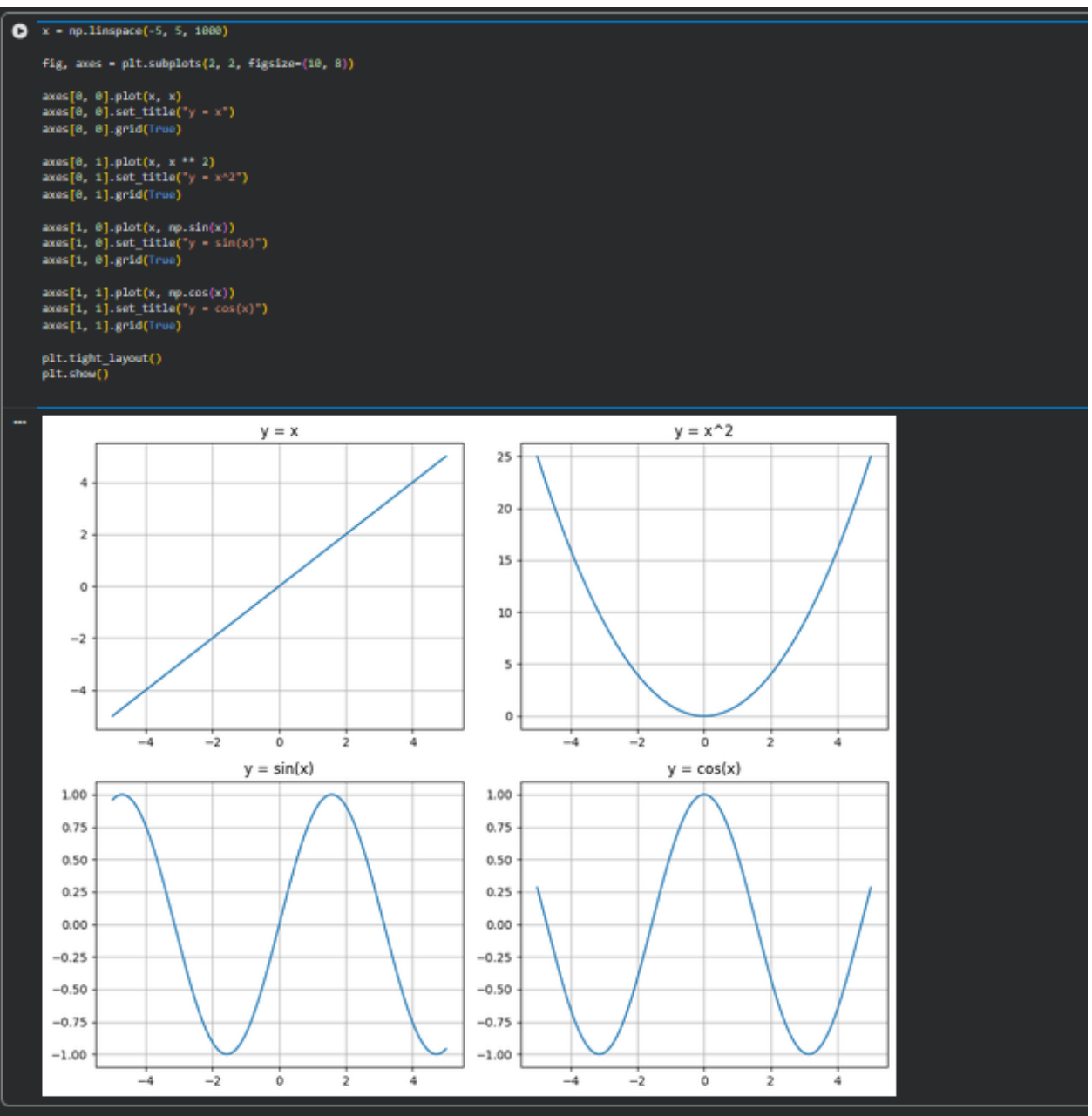


Рисунок 16. Код и результат выполнения 8 задания

```

np.random.seed(42)
matrix = np.random.rand(10, 10)

plt.figure(figsize=(6, 5))

image = plt.imshow(matrix, cmap="viridis", interpolation="nearest")
plt.colorbar(image)

plt.title("Тепловая карта случайной матрицы 10x10")

plt.show()

```

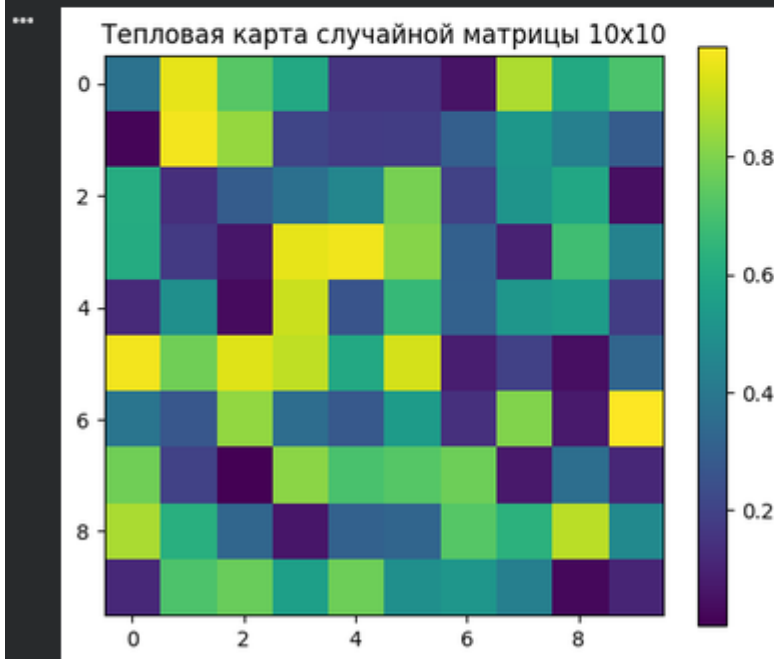


Рисунок 17. Код и результат выполнения 9 задания

Индивидуальные задания:

1. График температуры в течение недели

Постройте график изменения температуры воздуха за неделю. Данные:

- Дни недели: ['Пн', 'Вт', 'Ср', 'Чт', 'Пт', 'Сб', 'Вс']
- Температура (°C): [3, 5, 7, 10, 12, 8, 6]

Добавьте заголовок, подписи осей и сетку.


```

import matplotlib.pyplot as plt

days = ['Пн', 'Вт', 'Ср', 'Чт', 'Пт', 'Сб', 'Вс']
temperature = [3, 5, 7, 10, 12, 8, 6]

plt.figure(figsize=(8, 5))
plt.plot(days, temperature, marker='o')

plt.title("Изменение температуры воздуха за неделю")
plt.xlabel("Дни недели")
plt.ylabel("Температура (°C)")
plt.grid(True)

plt.show()

```



Рисунок 18. Результат выполнения кода

1. Продажи товаров в интернет-магазине

Интернет-магазин анализирует количество проданных единиц товаров за месяц:

- Категории товаров: ['Смартфоны', 'Ноутбуки', 'Планшеты', 'Наушники', 'Часы']
- Количество проданных единиц: [500, 300, 150, 400, 250]

Постройте вертикальную столбчатую диаграмму. Подпишите оси и добавьте заголовок.

```

import matplotlib.pyplot as plt

categories = ['Смартфоны', 'Ноутбуки', 'Планшеты', 'Наушники', 'Часы']
sales = [500, 300, 150, 400, 250]

plt.figure(figsize=(9, 5))
plt.bar(categories, sales)

plt.title("продажи товаров в интернет-магазине за месяц")
plt.xlabel("Категории товаров")
plt.ylabel("Количество проданных единиц")

plt.show()

```

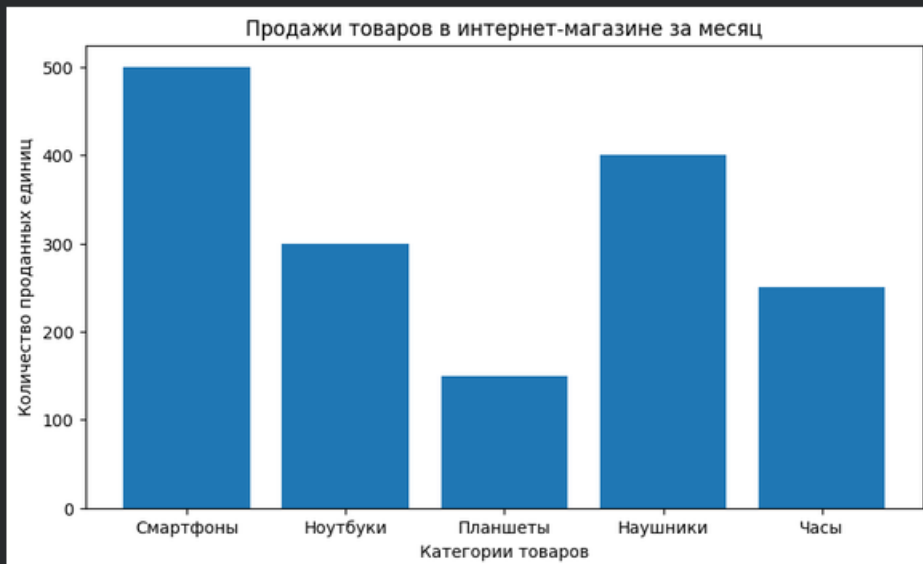


Рисунок 19. Результат выполнения кода

1. Площадь под параболой

Вычислите площадь под графиком функции:

$$f(x) = x^2$$

на интервале $[0, 3]$. Постройте график и закрасьте область под кривой.

```

import numpy as np
import matplotlib.pyplot as plt
import scipy.integrate as integrate

x = np.linspace(-1, 4, 1000)
y = x ** 2

x_fill = np.linspace(0, 3, 500)
y_fill = x_fill ** 2

area, _ = integrate.quad(lambda val: val**2, 0, 3)
print(f"Площадь под кривой: {area}")

plt.figure(figsize=(8, 6))
plt.plot(x, y, color="blue", linewidth=2)
plt.fill_between(x_fill, y_fill, color="lightblue", alpha=0.5)

plt.title("Визуализация определенного интеграла функции f(x) = x^2")
plt.xlabel("x")
plt.ylabel("y")
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(True)

plt.show()

```

*** Площадь под кривой: 9.000000000000002



Рисунок 20. Результат выполнения кода

Задачи на построение 3D-графиков с помощью Matplotlib

Во всех задачах требуется:

1. Построить трехмерный график функции $f(x, y)$ в заданных пределах.
2. Использовать библиотеку **Matplotlib** для визуализации.
3. Оформить график: добавить заголовок, подписи осей и цветовую карту (если уместно).

1. Параболоид вращения

Постройте 3D-график функции:

$$f(x, y) = x^2 + y^2$$

на интервале $x, y \in [-5, 5]$.

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
x_grid, y_grid = np.meshgrid(x, y)

z = x_grid ** 2 + y_grid ** 2

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection="3d")

surface = ax.plot_surface(
    x_grid,
    y_grid,
    z,
    cmap="viridis"
)

ax.set_title("Параболоид вращения  $f(x, y) = x^2 + y^2$ ")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")

fig.colorbar(surface, shrink=0.5, aspect=10)

plt.show()

```

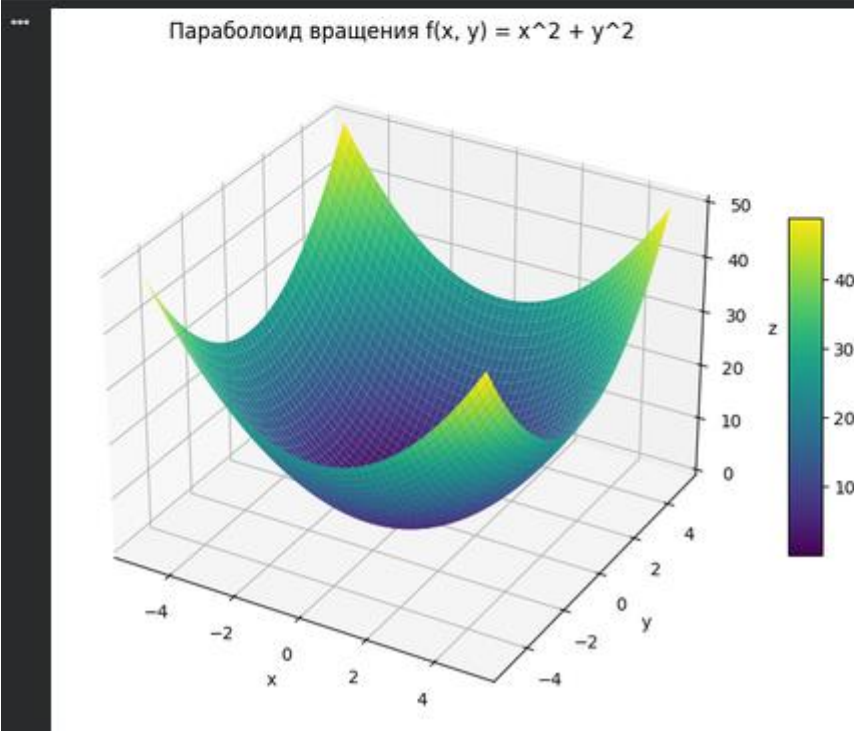


Рисунок 21. Результат выполнения кода

Ответы на контрольные вопросы:

1. Как осуществляется установка пакета matplotlib?

Команда `pip install matplotlib`. Если используется Anaconda или Miniconda, можно выполнить `conda install matplotlib`. В Jupyter Notebook установку также можно выполнить из ячейки командой `!pip install matplotlib`

или `!conda install matplotlib`. После установки библиотеку обычно подключают командой `import matplotlib.pyplot as plt`.

2. Какая "магическая" команда должна присутствовать в ноутбуках Jupyter для корректного отображения графиков matplotlib?

`%matplotlib inline`. Она обеспечивает вывод графиков непосредственно под ячейкой ноутбука. Для интерактивного отображения в некоторых средах может использоваться `%matplotlib notebook` или `%matplotlib widget`, но базовый и наиболее распространенный вариант - `%matplotlib inline`.

3. Как отобразить график с помощью функции `plot`?

Для отображения графика используется функция `plt.plot(x, y)`, где `x` - значения по горизонтальной оси, а `y` - соответствующие значения по вертикальной оси. После задания данных обычно вызывается `plt.show()`, чтобы вывести график на экран.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [1, 4, 9, 16]

plt.plot(x, y)
plt.show()
```

4. Как отобразить несколько графиков на одном поле?

Несколько графиков на одном поле можно построить несколькими последовательными вызовами `plt.plot()` до вызова `plt.show()`. Каждому графику можно задать свой цвет, стиль линии, маркеры и подпись для легенды.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y1 = [1, 4, 9, 16]
y2 = [1, 2, 3, 4]

plt.plot(x, y1, label='y = x2')
plt.plot(x, y2, label='y = x')
plt.legend()
plt.show()
```

5. Какой метод Вам известен для построения диаграмм категориальных данных?

Для построения диаграмм категориальных данных в matplotlib часто используется метод `plt.bar()`, который строит столбчатую диаграмму. Если категории нужно расположить по вертикальной оси, применяется `plt.barh()`. Также для категориальных данных могут использоваться круговые диаграммы `plt.pie()` и точечные диаграммы `plt.scatter()`, если необходимо сравнить значения по категориям.

6. Какие основные элементы графика Вам известны?

К основным элементам графика относятся область рисунка `Figure`, координатная область `Axes`, оси `X` и `Y`, шкалы и деления осей, подписи осей, заголовок, линии или маркеры данных, легенда, сетка, текстовые аннотации, цветовая шкала, границы области построения, а также дополнительные элементы: `errorbar`, заливка, маркеры, подписи точек и несколько подграфиков.

7. Как осуществляется управление текстовыми надписями на графике?

Управление текстовыми надписями выполняется с помощью функций `plt.title()`, `plt.xlabel()`, `plt.ylabel()`, `plt.text()` и `plt.annotate()`. Они позволяют задать заголовок, подписи осей, произвольный текст в координатах графика и аннотации со стрелками. Для настройки используются параметры `fontsize`, `color`, `fontweight`, `ha`, `va`, `rotation` и другие.

```
plt.title('Название графика')
plt.xlabel('Ось X')
plt.ylabel('Ось Y')
plt.text(2, 4, 'Текст на графике')
plt.annotate('Максимум', xy=(3, 9), xytext=(2, 12),
            arrowprops={'arrowstyle': '->'})
```

8. Как осуществляется управление легендой графика?

Легенда создается функцией `plt.legend()`. Чтобы элементы появились в легенде, при построении графика задают параметр `label`. Управлять легендой можно параметрами `loc`, `title`, `fontsize`, `frameon`, `ncol` и `bbox_to_anchor`. Например, `loc='best'` автоматически выбирает подходящее место, а `bbox_to_anchor` позволяет вынести легенду за пределы области графика.

```
plt.plot(x, y1, label='Первый график')
plt.plot(x, y2, label='Второй график')
plt.legend(loc='best', title='Обозначения')
```

9. Как задать цвет и стиль линий графика?

Цвет и стиль линий задаются параметрами `color`, `linestyle`, `linewidth` и `marker`. Также можно использовать короткую строковую запись, например `'r--'` означает красную пунктирную линию, `'bo'` - синюю линию с круглыми маркерами. Основные стили линий: `'-'` - сплошная, `'--'` - пунктирная, `'-.'` - штрихпунктирная, `'.'` - точечная.

```
plt.plot(x, y, color='red', linestyle='--', linewidth=2, marker='o')
```

10. Как выполнить размещение графика в разных полях?

Размещение графиков в разных полях выполняется с помощью подграфиков. Для этого используются функции `plt.subplot()` или `plt.subplots()`. Функция `plt.subplots(nrows, ncols)` создает сетку областей построения и возвращает объект `Figure` и массив `Axes`, на каждом из которых можно построить отдельный график.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(1, 2)
ax[0].plot(x, y1)
ax[0].set_title('Первый график')
ax[1].plot(x, y2)
ax[1].set_title('Второй график')
plt.tight_layout()
plt.show()
```

11. Как выполнить построение линейного графика с помощью matplotlib?

Линейный график строится функцией `plt.plot()` или методом `ax.plot()`. Необходимо подготовить массивы значений `x` и `y`, передать их в функцию, при необходимости настроить цвет, стиль линии, подписи осей, сетку и легенду, затем вывести график с помощью `plt.show()`.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)
```



```
y = np.sin(x)

plt.plot(x, y, label='sin(x)')
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)
plt.legend()
plt.show()
```

12. Как выполнить заливку области между графиком и осью?

Между двумя графиками?

Заливка области между графиком и осью выполняется функцией `plt.fill_between(x, y)`, которая закрашивает область между кривой `y` и горизонтальной осью или заданным уровнем. Для заливки между двумя графиками используется `plt.fill_between(x, y1, y2)`. Цвет, прозрачность и другие параметры задаются через `color` и `alpha`.

```
plt.plot(x, y1)
plt.fill_between(x, y1, 0, color='skyblue', alpha=0.4)

plt.plot(x, y1)
plt.plot(x, y2)
plt.fill_between(x, y1, y2, color='orange', alpha=0.3)
```

13. Как выполнить выборочную заливку, которая удовлетворяет некоторому условию?

Выборочная заливка выполняется с помощью параметра `where` функции `plt.fill_between()`. В этот параметр передается логическое условие, определяющее, на каких участках нужно выполнить заливку. Для корректного пересечения границ условия часто добавляют параметр `interpolate=True`.

```
plt.plot(x, y)
plt.fill_between(x, y, 0, where=(y > 0), color='green',
alpha=0.4, interpolate=True)
```

14. Как выполнить двухцветную заливку?

Двухцветная заливка выполняется двумя вызовами `plt.fill_between()` с разными условиями и разными цветами. Например, положительные значения можно закрасить зеленым цветом, а отрицательные - красным.

```
plt.plot(x, y, color='black')
```

```
plt.fill_between(x, y, 0, where=(y >= 0), color='green',  
alpha=0.4, interpolate=True)  
plt.fill_between(x, y, 0, where=(y < 0), color='red',  
alpha=0.4, interpolate=True)
```

15. Как выполнить маркировку графиков?

Маркировка графиков выполняется с помощью параметра `marker` в функции `plot` или `scatter`. Маркеры позволяют выделить отдельные точки данных. Можно задавать тип маркера, его размер, цвет, цвет границы и подпись в легенде. Распространенные маркеры: 'o' - круг, 's' - квадрат, '^' - треугольник, 'x' - крестик, '*' - звезда.

```
plt.plot(x, y, marker='o', markersize=6, markerfacecolor='white',  
markeredgecolor='blue')
```

16. Как выполнить обрезку графиков?

Обрезка графика обычно выполняется заданием пределов осей с помощью `plt.xlim()` и `plt.ylim()` или методов `ax.set_xlim()` и `ax.set_ylim()`. Таким образом можно показать только нужный диапазон значений. Также можно предварительно отфильтровать массивы данных по условию и построить график только для выбранного диапазона.

```
plt.plot(x, y)  
plt.xlim(0, 5)  
plt.ylim(-1, 1)  
plt.show()
```

17. Как построить ступенчатый график? В чем особенность ступенчатого графика?

Ступенчатый график строится функцией `plt.step()` или параметром `drawstyle='steps'` в `plt.plot()`. Его особенность состоит в том, что значение функции изменяется скачками, а между точками остается постоянным. Такой график удобен для отображения дискретных процессов, сигналов, уровней, тарифов и накопительных значений.

```
plt.step(x, y, where='post')  
plt.show()
```

18. Как построить стековый график? В чем особенность стекового графика?

Стековый график строится функцией `plt.stackplot()`. Его особенность в том, что несколько рядов данных отображаются друг над другом, и общая высота показывает суммарное значение. Такой график используется для анализа структуры целого и вклада отдельных компонентов во времени или по категориям.

```
plt.stackplot(x, y1, y2, y3, labels=['A', 'B', 'C'])
plt.legend(loc='upper left')
plt.show()
```

19. Как построить stem-график? В чем особенность stem-графика?

Stem-график строится функцией `plt.stem()`. Его особенность состоит в том, что каждое значение отображается маркером, соединенным с базовой линией вертикальным отрезком. Такой тип графика часто используется для дискретных сигналов, последовательностей и спектров.

```
plt.stem(x, y)
plt.show()
```

20. Как построить точечный график? В чем особенность точечного графика?

Точечный график строится функцией `plt.scatter()`. Его особенность заключается в отображении отдельных наблюдений в виде точек без обязательного соединения линиями. Он удобен для анализа зависимости между двумя переменными, выявления кластеров, выбросов и характера распределения данных. Размер и цвет точек можно задавать параметрами `s` и `c`.

```
plt.scatter(x, y, s=50, c='blue', alpha=0.7)
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

21. Как осуществляется построение столбчатых диаграмм с помощью matplotlib?

Столбчатые диаграммы строятся с помощью `plt.bar()` для вертикальных столбцов и `plt.barh()` для горизонтальных. В функцию передаются категории или позиции столбцов и соответствующие значения. Можно задавать цвет, ширину столбцов, подписи, легенду и значения ошибок.

```
categories = ['A', 'B', 'C']
values = [10, 15, 7]

plt.bar(categories, values, color='steelblue')
plt.xlabel('Категории')
plt.ylabel('Значения')
plt.show()
```

22. Что такое групповая столбчатая диаграмма? Что такое столбчатая диаграмма с `errorbar` элементом?

Групповая столбчатая диаграмма - это диаграмма, на которой для каждой категории отображается несколько столбцов, соответствующих разным группам или сериям данных. Для построения таких диаграмм столбцы смещают относительно центральной позиции категории.

Столбчатая диаграмма с `errorbar` элементом - это диаграмма, на которой вместе со столбцами отображаются интервалы ошибок или неопределенности значений. Они задаются параметром `yerr` или `xerr` в функции `plt.bar()`. Такие элементы помогают показать стандартное отклонение, стандартную ошибку или доверительный интервал.

```
import numpy as np

labels = ['A', 'B', 'C']
x_pos = np.arange(len(labels))
width = 0.35

plt.bar(x_pos - width/2, values1, width, label='Группа 1')
plt.bar(x_pos + width/2, values2, width, label='Группа 2',
yerr=errors)
plt.xticks(x_pos, labels)
plt.legend()
plt.show()
```

23. Как выполнить построение круговой диаграммы средствами matplotlib?

Круговая диаграмма строится функцией `plt.pie()`. В нее передаются значения долей, а также подписи `labels`. С помощью `autopct` можно отобразить проценты, `startangle` задает начальный угол, `explode` позволяет выделить отдельный сектор, а `shadow` включает тень.

```
values = [40, 30, 20, 10]
labels = ['A', 'B', 'C', 'D']

plt.pie(values, labels=labels, autopct='%1.1f%%',
startangle=90)
plt.axis('equal')
plt.show()
```

24. Что такое цветовая карта? Как осуществляется работа с цветовыми картами в matplotlib?

Цветовая карта, или `colormap`, - это правило сопоставления числовых значений с цветами. Она используется для отображения двумерных массивов, тепловых карт, поверхностей и точечных графиков, где цвет несет дополнительную информацию. В `matplotlib` цветовая карта задается параметром `cmap`, например `cmap='viridis', 'plasma', 'inferno', 'magma', 'gray', 'coolwarm'`. Цветовую шкалу можно добавить функцией `plt.colorbar()`.

```
plt.scatter(x, y, c=values, cmap='viridis')
plt.colorbar(label='Значение')
plt.show()
```

25. Как отобразить изображение средствами matplotlib?

Изображение отображается функцией `plt.imshow()`. Изображение можно загрузить с помощью `plt.imread()` или передать в виде массива `NumPy`. Для скрытия координатных осей используется `plt.axis('off')`. Для полутоновых изображений можно указать `cmap='gray'`.

```
img = plt.imread('image.png')
plt.imshow(img)
plt.axis('off')
plt.show()
```

26. Как отобразить тепловую карту средствами matplotlib?

Тепловая карта отображает значения двумерного массива с помощью цвета. В matplotlib ее можно построить функцией plt.imshow() или plt.pcolormesh(). Обычно задают цветовую карту cmap, добавляют цветовую шкалу plt.colorbar(), а при необходимости подписывают оси и ячейки.

```
import numpy as np

data = np.random.rand(5, 5)
plt.imshow(data, cmap='viridis')
plt.colorbar(label='Интенсивность')
plt.show()
```

27. Как выполнить построение линейного 3D-графика с помощью matplotlib?

Для построения линейного 3D-графика используется модуль mpl_toolkits.mplot3d. Нужно создать трехмерную область построения с параметром projection='3d', после чего вызвать метод ax.plot(x, y, z). Такой график отображает линию в трехмерном пространстве.

```
import numpy as np
import matplotlib.pyplot as plt

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
t = np.linspace(0, 10, 100)
x = np.sin(t)
y = np.cos(t)
z = t

ax.plot(x, y, z)
plt.show()
```

28. Как выполнить построение точечного 3D-графика с помощью matplotlib?

Точечный 3D-график строится методом ax.scatter(x, y, z) на трехмерной области построения. Он используется для отображения набора точек в пространстве. Можно задавать цвет, размер, прозрачность и цветовую карту.

```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(x, y, z, c=z, cmap='viridis', s=30)
```

```
plt.show()
```

29. Как выполнить построение каркасной поверхности с помощью matplotlib?

Каркасная поверхность строится методом `ax.plot_wireframe(X, Y, Z)`. Для этого сначала создают сетку координат `X` и `Y` с помощью `np.meshgrid()`, затем вычисляют значения `Z`. Каркасная поверхность показывает форму трехмерной функции с помощью линий сетки без сплошной заливки.

```
x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_wireframe(X, Y, Z)
plt.show()
```

30. Как выполнить построение трехмерной поверхности с помощью matplotlib?

Трехмерная поверхность строится методом `ax.plot_surface(X, Y, Z)`. Как и для каркасной поверхности, сначала создается сетка координат через `np.meshgrid()`, затем вычисляется массив `Z`. В отличие от каркасного графика, поверхность отображается сплошной заливкой, цвет которой можно задать с помощью цветовой карты `cm`.

```
x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, cmap='viridis')
plt.show()
```

Вывод: в ходе работы были успешно освоены возможности библиотеки `matplotlib` языка `Python`.